

Laporan Praktikum Modul IV:
STATE HOISTING, PERUBAHAN KONFIGURASI, DAN SIDE-
EFFECT

Tugas Mata Pengembangan Aplikasi Mobile

Dosen Pengampu :
Muhammad Aminul Akbar, S.Kom., M.T



Disusun Oleh:

Nazwa Hafidzah

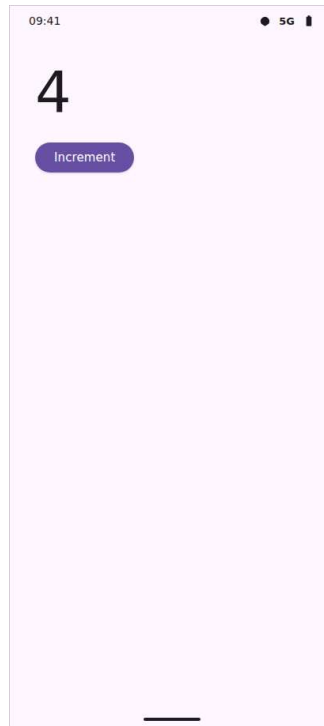
245150407111023

SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

3. PEMBAHASAN PERCOBAAN & JAWABAN LEMBAR KERJA

3.1 Percobaan 4.5.1: State Hoisting

A. Tahap 1: Kode Awal (Stateful Counter)



Gambar 1.1: Tampilan setelah tombol Increment diklik beberapa kali (angka menjadi 4)

a. Apa yang terjadi setiap kali tombol 'Increment' diklik?

Jawaban: Setiap kali tombol 'Increment' diklik, angka yang tampil di layar bertambah satu secara berurutan (0 jadi 1, 2, 3, dan seterusnya).

b. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Karena variabel number dibuat menggunakan `mutableStateOf(0)`. Begitu tombol diklik dan menjalankan `number++`, Jetpack Compose langsung mendeteksi ada perubahan data pada state tersebut, lalu otomatis menggambar ulang (recomposition) teks angka di layar.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka?

Jawaban: Variabel state number, yang dideklarasikan dengan `'var number by remember { mutableStateOf(0) }'`.

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button?

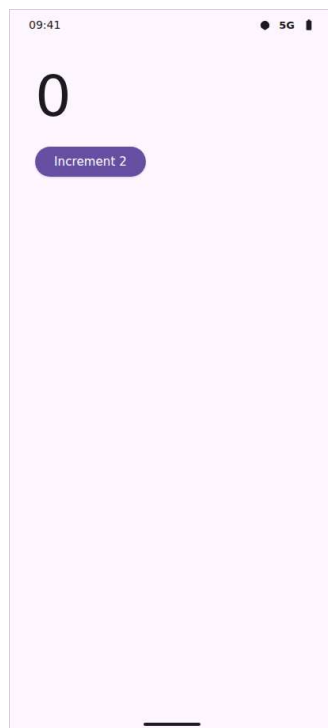
Jawaban: Variabel state label, yang dideklarasikan dengan 'val label by remember { mutableStateOf("Increment") }'.

B. Tahap 2: Modifikasi Penambahan 2 dan Label 'Increment 2'

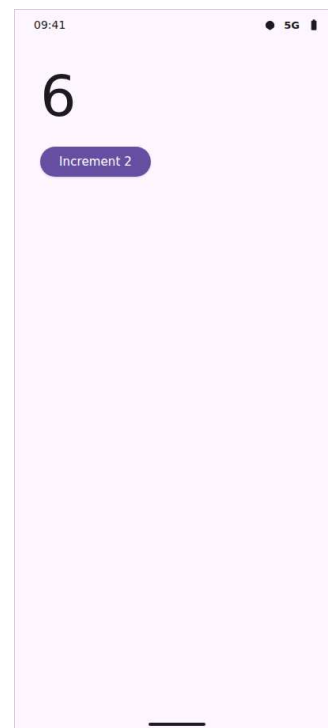
Pertanyaan 2a: Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Jawaban: Perubahan dilakukan langsung di dalam badan fungsi Composable CounterScreen pada dua baris:

1. Nilai variabel label diganti menjadi: val label by remember { mutableStateOf("Increment 2") }
2. Event klik pada Button diubah menjadi: Button(onClick = { number += 2 })



Gambar 1.2: Sebelum diklik (nilai awal 0)



Gambar 1.3: Sesudah diklik 3 kali (nilai bertambah menjadi 6)

C. Tahap 3 & 4: Menerapkan State Hoisting pada CounterScreen

```
// CounterScreen setelah state hoisting (stateless & reusable)
@Composable
fun CounterScreen(
    modifier: Modifier = Modifier,
```

```

        number: Int,
        label: String,
        onClick: () -> Unit
    ) {
        Column(modifier = modifier) {
            Text(text = "$number", fontSize = 72.sp)
            Button(onClick = onClick) {
                Text(text = label)
            }
        }
    }

// Pemanggilan di fungsi onCreate():
var number by remember { mutableStateOf(0) }
CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ }
)

```

a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti tahap sebelumnya?

Jawaban: Ya, program tetap berjalan lancar dan tampilan visual serta tombolnya bekerja sama persis.

b. Di bagian kode program manakah saat ini proses increment dilakukan?

Jawaban: Logika penambahan sekarang dipindah ke luar fungsi CounterScreen, tepatnya di fungsi onCreate() Activity lewat parameter lambda callback 'onClick = { number++ }'.

c. Di bagian kode program manakah saat ini state Composable Text dan Button dideklarasikan?

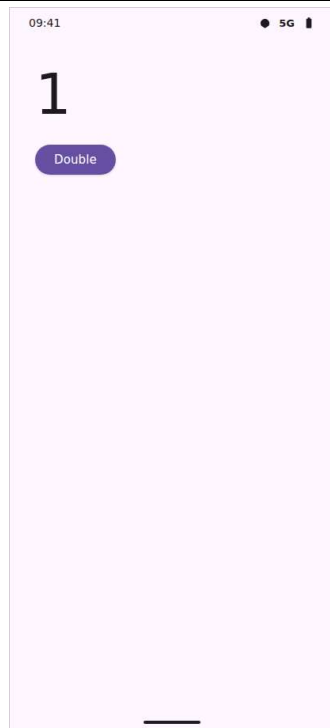
Jawaban: Variabel state number sekarang berada di fungsi onCreate() Activity (var number by remember { mutableStateOf(0) }), sedangkan teks tombol dikirim langsung

sebagai parameter string biasa ("Increment"). Fungsi CounterScreen sekarang murni menjadi stateless.

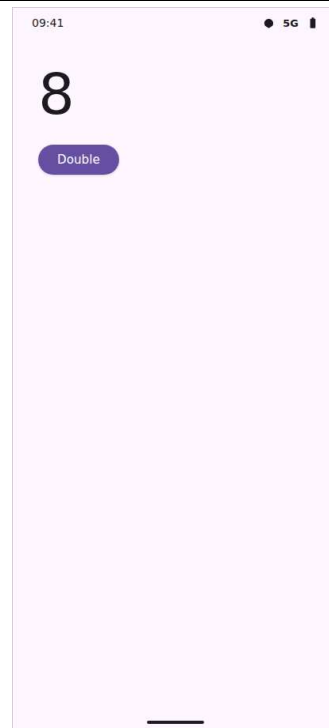
D. Tahap 5, 6, & 7: Perubahan Kebutuhan (Tombol 'Double' & Nilai Awal 1)

Pertanyaan 5d: Tuliskan kode program deklarasi state dan pemanggilan CounterScreen setelah perubahan kebutuhan:

```
// Di dalam fungsi onCreate():  
var number by remember { mutableStateOf(1) }  
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```



Gambar 1.4: Sebelum diklik (nilai awal 1)



Gambar 1.5: Sesudah tombol Double diklik 3 kali (1 -> 2 -> 4 -> 8)

6b. Di bagian manakah dari kode program modifikasi tahap nomor 7 dan 8 dilakukan?

Jawaban: Semua perubahan hanya dilakukan di fungsi pemanggil pada onCreate().

6c. Apakah ada perubahan yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri?

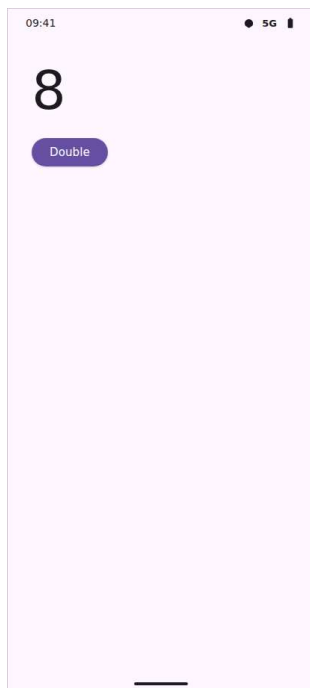
Jawaban: Tidak ada sama sekali. Kode fungsi CounterScreen tetap utuh tanpa diubah sedikit pun.

7. Apa arti dari temuan ini terkait manfaat state hoisting dalam perubahan kebutuhan fungsional?

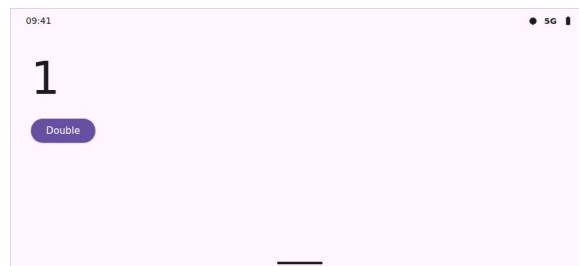
Jawaban: State hoisting berhasil memisahkan tampilan UI dari logika pengelolaan datanya. Karena CounterScreen dibuat stateless, saat aturan bisnis berubah (misal dari tambah satu menjadi kali dua), kita cukup mengubah logika di pemanggilnya saja tanpa perlu mengutak-atik kode komponen UI tersebut.

3.2 Percobaan 4.5.2: Perubahan Konfigurasi (Configuration Change)

A. Pengujian Menggunakan remember



Gambar 2.1: Posisi Portrait (nilai 8)



Gambar 2.2: Posisi Landscape (nilai kembali ke 1)

Pertanyaan 2b: Berapa angka yang ditampilkan di posisi Landscape? Mengapa terjadi demikian?

Jawaban: Angka yang tampil kembali menjadi 1. Penyebabnya adalah saat layar diputar, sistem Android menghancurkan Activity (onDestroy) dan membentuk Activity baru (onCreate). Konstruk remember hanya menyimpan data di memori selama Activity yang

bersangkutan masih aktif. Karena Activity di-restart, data lama terhapus dan nilai variabel langsung kembali ke angka awal (1).

B. Pengujian Menggunakan rememberSaveable

3b. Berapa angka yang ditampilkan di posisi Landscape setelah menggunakan rememberSaveable? Mengapa?

Jawaban: Angka yang tampil tetap 16 (tidak reset). Hal ini karena rememberSaveable otomatis menitipkan nilai state ke Bundle sistem Android saat Activity dihancurkan. Begitu Activity baru selesai dibuat sehabis rotasi, nilai 16 tersebut langsung diambil kembali dari Bundle.

3c. Apa fungsi rememberSaveable yang membedakannya dari konstruk remember?

Jawaban: Konstruk remember hanya menjaga data selama proses recomposition pada instance Activity yang sedang berjalan. Sedangkan rememberSaveable mampu menjaga data tetap tersimpan saat Activity di-restart akibat rotasi layar, pergantian tema gelap/terang, maupun saat aplikasi dimatikan sementara oleh sistem di latar belakang.

4. Berikan contoh kasus nyata lain di mana rememberSaveable tepat digunakan serta manfaatnya:

Jawaban: Contoh penggunaan nyata:

1. Formulir Isian: Menyimpan ketikan teks nama, alamat, atau nominal pembayaran agar tidak mendadak kosong saat pengguna tidak sengaja memutar ponsel.
2. Status Tampilan: Menyimpan pilihan tab aktif, filter pencarian toko online, dan centang checkbox.
3. Posisi Daftar: Menjaga posisi scroll pada daftar berita atau produk.

Manfaat: Mencegah pengguna kesal karena tidak perlu mengetik ulang data dari awal setiap kali orientasi layar HP berubah.

3.3 Percobaan 4.5.3: Side-effect dengan LaunchedEffect

A. Mengaitkan Snackbar dan LaunchedEffect(number)

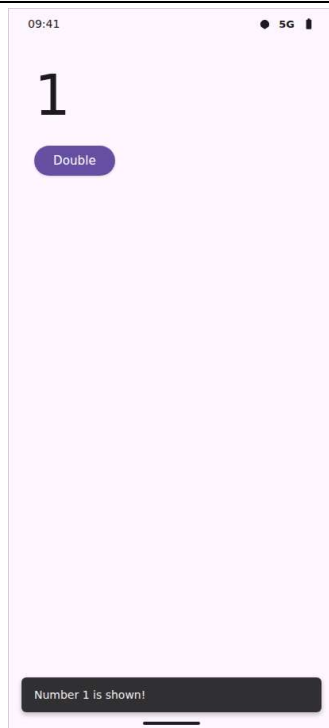
```
val snackbarHostState = remember { SnackbarHostState() }  
Scaffold(  
    modifier = Modifier.fillMaxSize(),  
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }  
) { innerPadding ->
```

```

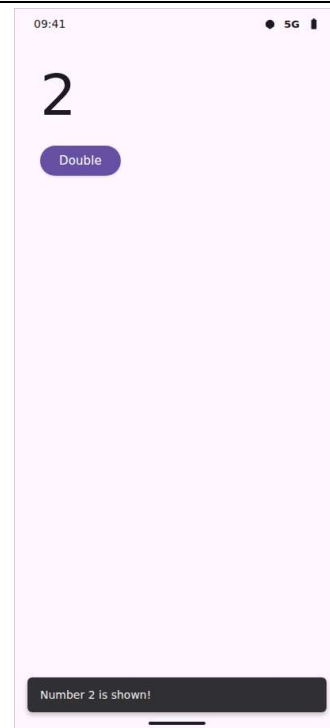
Column(modifier = Modifier.padding(innerPadding)) {
    CounterScreen(
        modifier = Modifier.padding(32.dp),
        number = number,
        label = "Double",
        onClick = { number *= 2 }
    )
}
}

LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}

```



Gambar 3.1: Pertama kali dibuka (pesan: 'Number 1 is shown!')



Gambar 3.2: Setelah diklik 1 kali (angka 2, pesan: 'Number 2 is shown!')

1a. Pada saat pertama kali program dijalankan tanpa mengklik tombol, apa yang ditampilkan?

Jawaban: Di layar langsung muncul angka 1, tombol 'Double', dan pesan Snackbar di bagian bawah bertuliskan 'Number 1 is shown!'.

1b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa interaksi?

Jawaban: Karena `LaunchedEffect` pasti dijalankan sekali saat komponen tampilan pertama kali muncul di layar (initial composition). Saat itu nilai awal `number` bernilai 1, sehingga coroutine langsung mengeksekusi perintah `showSnackBar`.

1c. Apa yang memicu pesan pada SnackBar tersebut ditampilkan? Pada bagian kode program mana?

Jawaban: Dipicu oleh baris kode `LaunchedEffect(number) {
 snackbarHostState.showSnackBar("Number $number is shown!") }`.

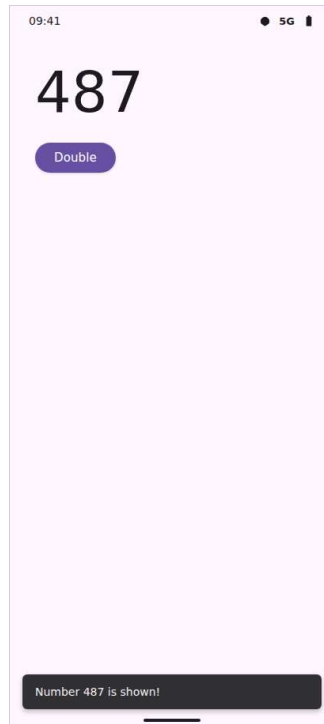
2a & 2b. Klik tombol 'Double' satu kali: Apa yang terjadi dan mengapa?

Jawaban: Angka di layar berubah jadi 2 dan muncul notifikasi SnackBar baru bertuliskan 'Number 2 is shown!'. Ini terjadi karena variabel `number` dijadikan key pada `LaunchedEffect(number)`. Saat nilainya berganti dari 1 ke 2, Compose mendeteksi pergantian key tersebut, lalu membatalkan coroutine lama dan menjalankan coroutine baru membawa nilai terbaru.

B. Simulasi Ambil Data Asinkron dengan `LaunchedEffect(Unit)`

```
// File: Repo.kt
class Repo {
    companion object {
        suspend fun getData(): Int {
            delay(2000) // simulasi jeda jaringan 2 detik
            return Random.nextInt(100, 1000)
        }
    }
}

// Di dalam onCreate() Activity:
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```



Gambar 3.3: Setelah 2 detik, angka otomatis berubah (contoh: 487) dan memicu Snackbar baru

1a. Pada saat pertama kali program dijalankan, apa yang ditampilkan di layar?

Jawaban: Selama 2 detik pertama layar masih menampilkan angka 1 dan Snackbar awal bertuliskan 'Number 1 is shown!'.

1b. Tunggu selama dua detik, apa yang tampil di layar?

Jawaban: Tepat setelah 2 detik, angka di layar otomatis berubah menjadi angka acak antara 100 sampai 1000 (misalnya 487) dan muncul Snackbar baru bertuliskan 'Number 487 is shown!'.

1c. Apa yang memicu perubahan angka dan pesan pada Snackbar tersebut? Pada kode mana?

Jawaban: Perubahan angka dipicu oleh baris `'number = Repo.getData()'` di dalam `LaunchedEffect(Unit)`. Saat fungsi suspend tersebut selesai setelah 2 detik, variabel `number` diisi nilai baru. Perubahan pada `number` ini kemudian memicu `LaunchedEffect(number)` untuk memunculkan pesan Snackbar yang baru.

2a & 2b. Analisis Perbedaan `LaunchedEffect(state)` vs `LaunchedEffect(Unit)`:

Jawaban: `LaunchedEffect(Unit)` dijalankan hanya satu kali ketika komponen pertama kali dibuat atau ditampilkan. Hal ini karena parameter `key` yang digunakan adalah `Unit`, yaitu nilai konstan yang tidak pernah berubah, sehingga `Compose` tidak akan

menjalankan ulang coroutine tersebut. Penggunaan `LaunchedEffect(Unit)` tepat untuk proses yang hanya perlu dilakukan sekali saat halaman pertama kali dibuka, seperti melakukan inisialisasi awal, mengambil data dari API, atau mengambil data dari database.

2. `LaunchedEffect(state)`:

`LaunchedEffect` dengan key berupa variabel state akan dijalankan saat komponen pertama kali dibuka dan akan dijalankan kembali setiap kali nilai variabel state tersebut berubah. Hal ini terjadi karena `Compose` mendeteksi adanya perubahan pada nilai key, sehingga coroutine yang sedang berjalan akan dibatalkan dan kemudian menjalankan coroutine baru dengan nilai terbaru. Penggunaan `LaunchedEffect` seperti ini tepat untuk aksi reaktif yang bergantung pada perubahan data pada UI, misalnya menampilkan animasi atau Snackbar notifikasi ketika data berubah.

4. KESIMPULAN

State hoisting memisahkan tampilan UI dari logika pengelolaan data, sehingga fungsi `Composable` menjadi lebih modular, mudah diuji, dan dapat digunakan kembali tanpa bergantung pada logika tertentu. `rememberSaveable` berfungsi menjaga data input atau status UI agar tidak hilang ketika terjadi perubahan konfigurasi seperti rotasi layar dengan menyimpannya ke Bundle sistem, berbeda dengan `remember` yang dapat kehilangan state ketika Activity dibuat ulang. Sementara itu, `LaunchedEffect` menyediakan cara yang aman untuk menjalankan coroutine dan proses asinkron di Jetpack Compose karena siklus hidupnya dikelola secara otomatis dan coroutine akan dibatalkan ketika composable tersebut keluar dari komposisi.